

LAPORAN PRAKTIKUM SISTEM OPERASI

Dosen Pengampu :

TRIAWAN ADI CAHYANTO M.Kom



Disusun Oleh:

Pradana Bagus Susilo (2410651007)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tujuan

1. Memahami Konsep Dasar Sistem Operasi

- Menjelaskan fungsi utama sistem operasi, seperti manajemen proses, memori, dan perangkat keras.
- Membedakan antara kernel, shell, dan distribusi Linux (misalnya, Ubuntu, CentOS).
- Mengidentifikasi komponen utama Linux, termasuk file system (ext4, Btrfs) dan boot process.

2. Menguasai Command Line Interface (CLI)

- Menggunakan perintah dasar seperti ls, cd, mkdir, cp, mv, dan rm untuk navigasi dan manipulasi file.
- Memahami penggunaan pipes, redirection, dan wildcards untuk efisiensi kerja.
- Menjalankan perintah dengan opsi lanjutan, seperti grep, find, dan sed untuk pencarian dan pengeditan teks.

3. Mengelola File dan Direktori

- Mengatur izin file dan direktori menggunakan chmod, chown, dan chgrp.
- Memahami struktur file system Linux (/root, /home, /etc, dll.) dan mounting perangkat.
- Menggunakan alat seperti tar, gzip, dan rsync untuk kompresi dan sinkronisasi file.

4. Menginstal dan Mengkonfigurasi Software

- Menggunakan package manager seperti apt (Debian-based) atau yum/dnf (Red Hat-based) untuk instalasi aplikasi.
- Mengkompilasi software dari source code menggunakan make dan configure.
- Mengelola dependensi dan repositori perangkat lunak.

5. Mengelola Proses dan Pengguna

- Memantau dan mengontrol proses dengan ps, top, kill, dan nice.
- Membuat dan mengelola akun pengguna serta grup menggunakan useradd, usermod, dan passwd.
- Mengonfigurasi layanan sistem seperti systemd untuk startup otomatis.

6. Dasar-Dasar Jaringan dan Keamanan

- Mengkonfigurasi jaringan dengan ifconfig atau ip, termasuk pengaturan IP, DNS, dan firewall (iptables atau firewalld).
- Memahami protokol dasar seperti SSH untuk akses jarak jauh dan SCP untuk transfer file.

Topik 1: Threads & Concurrency

Source Kode:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Fungsi yang akan dijalankan oleh thread
void* cetak_pesan(void* arg) {
    char* pesan = (char*) arg;

    for (int i = 1; i <= 5; i++) {
        printf("%s - hitungan ke-%d\n", pesan, i);
        sleep(1); // Tidur 1 detik
    }

    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== Program Dimulai ===\n\n");

    // Membuat 2 thread
    pthread_create(&thread1, NULL, cetak_pesan, "Thread 1");
    pthread_create(&thread2, NULL, cetak_pesan, "Thread 2");

    // Menunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== Program Selesai ===\n");

    return 0;
}
```

Output:

```
pradana@Bagus:~$ nano thread_sederhana.c
pradana@Bagus:~$ gcc thread_sederhana.c -o thread_sederhana -pthread
pradana@Bagus:~$ ./thread_sederhana
=== Program Dimulai ===

Thread 1 - hitungan ke-1
Thread 2 - hitungan ke-1
Thread 2 - hitungan ke-2
Thread 1 - hitungan ke-2
Thread 2 - hitungan ke-3
Thread 1 - hitungan ke-3
Thread 2 - hitungan ke-4
Thread 1 - hitungan ke-4
Thread 2 - hitungan ke-5
Thread 1 - hitungan ke-5

=== Program Selesai ===
pradana@Bagus:~$ |
```

Analisis Hasil:

Program ini tampaknya menunjukkan konsep dasar concurrency :

- Dua proses mengakses memori CPU yang sama dengan berbagi memori data yang sama,
- Dua thread bekerja pada satu proses, eksekusi yang sama secara bersamaan.

Output kedua thread bervariasi dan bercampur dan bukan dalam urutan yang ditentukan.

Program hanya selesai setelah kedua thread selesai.

Topik 2: Thread dengan Shared Memory

Source Kode:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // Variabel yang dibagi bersama

void* tambah_counter(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; // Menambah counter
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("Counter awal: %d\n", counter);

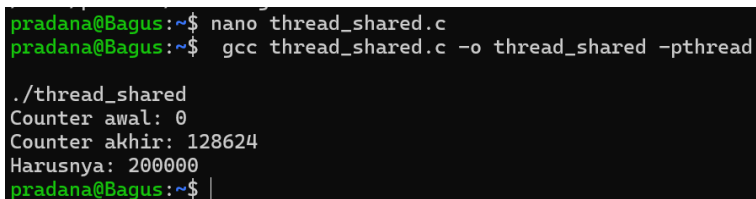
    // Buat 2 thread yang menambah counter
    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    // Tunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    return 0;
}
```

Apa Yang Terjadi?



```
pradana@Bagus:~$ nano thread_shared.c
pradana@Bagus:~$ gcc thread_shared.c -o thread_shared -pthread

./thread_shared
Counter awal: 0
Counter akhir: 128624
Harusnya: 200000
pradana@Bagus:~$ |
```

Hasil akhirnya tidak selalu 200000, tapi biasanya lebih kecil dan bervariasi setiap dijalankan. Hal ini terjadi karena dua thread menambah variabel counter secara bersamaan tanpa sinkronisasi, sehingga sebagian operasi penambahan saling menimpa (race condition). Akibatnya, ada beberapa kenaikan counter yang “hilang”, jadi nilai akhir tidak sesuai harapan.

Topik 3: Mengatasi Race Condition dengan Mutex

Source Kode:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock; // Variabel mutex (kunci)

void* tambah_counter(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock); // Kunci pintu
        counter++;                 // Area aman
        pthread_mutex_unlock(&lock); // Buka kunci
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    // Inisialisasi mutex
    pthread_mutex_init(&lock, NULL);

    printf("Counter awal: %d\n", counter);

    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    // Hapus mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```

Apa Yang Terjadi?

```
pradana@Bagus:~$ nano thread_mutex.c
pradana@Bagus:~$ gcc thread_mutex.c -o thread_mutex -pthread
./thread_mutex
Counter awal: 0
Counter akhir: 200000
Harusnya: 200000
pradana@Bagus:~$ |
```

Akhirnya, hasil program selalu sama yaitu 200000, namun ini terjadi karena kedua thread tidak dapat mengakses counter secara bersamaan dikarenakan penggunaan mutex. Masing-masing thread harus menunggu penyimpanan sesi sebelum mengizinkan thread lainnya untuk menambahkan nilai itu, sehingga tidak akan ada race condition atau iterasi yang tidak terhitung kenaikan; semua hitungan valid

Topik 4: Thread Pool Sederhana

Source Kode:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define JUMLAH_THREAD 3
#define JUMLAH_TUGAS 10

void* kerjakan_tugas(void* arg) {
    int id = *((int*) arg);

    printf("Thread %d: Mulai mengerjakan tugas\n", id);
    sleep(2); // Simulasi pekerjaan
    printf("Thread %d: Selesai mengerjakan tugas\n", id);

    return NULL;
}

int main() {
    pthread_t threads[JUMLAH_THREAD];
    int thread_ids[JUMLAH_THREAD];

    printf("=== Thread Pool dengan %d threads ===\n\n", JUMLAH_THREAD);

    // Buat pool of threads
    for (int i = 0; i < JUMLAH_THREAD; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, kerjakan_tugas, &thread_ids[i]);
    }

    // Tunggu semua thread selesai
    for (int i = 0; i < JUMLAH_THREAD; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua tugas selesai ===\n");

    return 0;
}
```


Apa Yang Terjadi?

```
pradana@Bagus:~$ nano thread_pool.c
pradana@Bagus:~$ gcc thread_pool.c -o thread_pool -pthread
./thread_pool
=== Thread Pool dengan 3 threads ===

Thread 1: Mulai mengerjakan tugas
Thread 2: Mulai mengerjakan tugas
Thread 3: Mulai mengerjakan tugas
Thread 3: Selesai mengerjakan tugas
Thread 1: Selesai mengerjakan tugas
Thread 2: Selesai mengerjakan tugas

=== Semua tugas selesai ===
pradana@Bagus:~$ |
```

Ketika dijalankan, program akan membuat 3 thread (`JUMLAH_THREAD = 3`).

Setiap thread akan menampilkan pesan bahwa dia mulai mengerjakan tugas, lalu tidur selama 2 detik (sleep) untuk mensimulasikan pekerjaan, kemudian menampilkan pesan bahwa dia selesai.

Topik 5: Data Parallelism

Source Kode:

```
#include <stdio.h>
#include <pthread.h>

#define UKURAN_ARRAY 1000
#define JUMLAH_THREAD 4

int array[UKURAN_ARRAY];
int jumlah_parsial[JUMLAH_THREAD];

typedef struct {
    int id;
    int mulai;
    int akhir;
} ThreadData;

void* hitung_jumlah(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int sum = 0;

    for (int i = data->mulai; i < data->akhir; i++) {
        sum += array[i];
    }

    jumlah_parsial[data->id] = sum;
    printf("Thread %d: Menghitung indeks %d-%d, hasil = %d\n",
        data->id, data->mulai, data->akhir - 1, sum);

    return NULL;
}

int main() {
    pthread_t threads[JUMLAH_THREAD];
    ThreadData thread_data[JUMLAH_THREAD];

    // Isi array dengan angka 1-1000
    for (int i = 0; i < UKURAN_ARRAY; i++) {
        array[i] = i + 1;
    }

    printf("=== Data Parallelism: Menjumlahkan array ===\n\n");

    int bagian = UKURAN_ARRAY / JUMLAH_THREAD;
```

```

// Buat thread untuk memproses bagian array
for (int i = 0; i < JUMLAH_THREAD; i++) {
    thread_data[i].id = i;
    thread_data[i].mulai = i * bagian;
    thread_data[i].akhir = (i + 1) * bagian;

    pthread_create(&threads[i], NULL, hitung_jumlah, &thread_data[i]);
}

// Tunggu semua thread selesai
for (int i = 0; i < JUMLAH_THREAD; i++) {
    pthread_join(threads[i], NULL);
}

// Jumlahkan hasil dari semua thread
int total = 0;
for (int i = 0; i < JUMLAH_THREAD; i++) {
    total += jumlah_parsial[i];
}

printf("\n=== Total keseluruhan: %d ===\n", total);
printf("Verifikasi: 1+2+...+1000 = %d\n", (UKURAN_ARRAY * (UKURAN_ARRAY +
1)) / 2);

return 0;
}

```

Apa yang Terjadi:

```

pradana@Bagus:~$ nano data_parallel.c
pradana@Bagus:~$ gcc data_parallel.c -o data_parallel -pthread

./data_parallel
=== Data Parallelism: Menjumlahkan array ===

Thread 0: Menghitung indeks 0-249, hasil = 31375
Thread 3: Menghitung indeks 750-999, hasil = 218875
Thread 1: Menghitung indeks 250-499, hasil = 93875
Thread 2: Menghitung indeks 500-749, hasil = 156375

=== Total keseluruhan: 500500 ===
Verifikasi: 1+2+...+1000 = 500500
pradana@Bagus:~$

```

Dari hasil tersebut terlihat bahwa setiap thread mengcalculusikan beberapa elemen di array, tetapi hasil dari semua thread dijumlahkan ke dalam satu nilai total. Hasil total yang diperoleh harus sama dengan perhitungan $1 + 2 + \dots + 1000$, yaitu 500500. Selain itu, setiap thread menampilkan rentang indeks yang dia proses serta hasil dari penjumlahannya. Tapi urutan tampil tersebut dapat bervariasi karena beberapa thread dieksekusi pada saat yang sama. Setelah semua thread eksekusi dilakukan maka output program adalah persekutuan dari hasil benar yang sama dengan nilai verifikasi.

Topik 6: Mendeteksi Jumlah CPU Core

Source Kode:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* tugas_berat(void* arg) {
    int id = *((int*) arg);
    long hasil = 0;

    printf("Thread %d: Memulai komputasi...\n", id);

    // Simulasi komputasi berat
    for (long i = 0; i < 1000000000; i++) {
        hasil += i;
    }

    printf("Thread %d: Selesai!\n", id);
    return NULL;
}

int main() {
    // Deteksi jumlah CPU core
    int jumlah_core = sysconf(_SC_NPROCESSORS_ONLN);

    printf("=== Informasi Sistem ===\n");
    printf("Jumlah CPU core: %d\n\n", jumlah_core);

    pthread_t threads[jumlah_core];
    int thread_ids[jumlah_core];

    printf("Membuat %d thread (sesuai jumlah core)...\n\n", jumlah_core);

    // Buat thread sebanyak jumlah core
    for (int i = 0; i < jumlah_core; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, tugas_berat, &thread_ids[i]);
    }

    // Tunggu semua thread selesai
    for (int i = 0; i < jumlah_core; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua thread selesai ===\n");
}
```

```

    return 0;
}

```

Apa Yang Terjadi?

Terminal 1:

```

Membuat 8 thread (sesuai jumlah core)...

Thread 1: Memulai komputasi...
Thread 3: Memulai komputasi...
Thread 2: Memulai komputasi...
Thread 4: Memulai komputasi...
Thread 5: Memulai komputasi...
Thread 6: Memulai komputasi...
Thread 7: Memulai komputasi...
Thread 8: Memulai komputasi...
Thread 7: Selesai!
Thread 2: Selesai!
Thread 1: Selesai!
Thread 4: Selesai!
Thread 8: Selesai!
Thread 5: Selesai!
Thread 6: Selesai!
Thread 3: Selesai!

=== Semua thread selesai ===
pradana@Bagus:~$

```

Terminal 2:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU
90	root	20	0	25268	6272	4864	S	0.3
1	root	20	0	21812	11980	8908	S	0.0
2	root	20	0	3072	1664	1664	S	0.0
6	root	20	0	3072	1792	1792	S	0.0
45	root	19	-1	66820	15200	14304	S	0.0
166	systemd+	20	0	21456	12544	10496	S	0.0
167	systemd+	20	0	91024	7552	6784	S	0.0
187	root	20	0	4236	2432	2304	S	0.0
188	message+	20	0	9624	4864	4480	S	0.0
201	root	20	0	17964	8192	7296	S	0.0
211	root	20	0	1755840	12288	10496	S	0.0
218	syslog	20	0	222508	5504	4480	S	0.0
224	root	20	0	3160	1920	1792	S	0.0
230	root	20	0	3116	1920	1792	S	0.0
239	root	20	0	107032	22016	12928	S	0.0
331	root	20	0	3076	896	896	S	0.0
333	root	20	0	3092	1152	1024	S	0.0
337	pradana	20	0	6068	4992	3456	S	0.0
340	root	20	0	6820	4352	3712	S	0.0
393	pradana	20	0	20316	11136	9088	S	0.0
395	pradana	20	0	21152	3520	1792	S	0.0
420	pradana	20	0	6072	4992	3456	S	0.0
842	root	20	0	3076	1028	896	S	0.0

Untuk dijalankan, dari atas, program memberi tahu berapa jumlah core di si komputer, lalu membuat thread sebanyak itu. Lalu tiap thread melakukan perhitungan besar secara bersamaan. “Memulai komputasi....” dan “Selesai!” muncul tidak berurutan. Barangkali semacam paralelisme. Setelah semua selesai: “Semua thread selesai”

Tugas 1

Source Kode:

```
#include <stdio.h>    // Digunakan untuk fungsi input-output (printf, scanf)
#include <pthread.h>   // Library untuk menggunakan thread
#include <stdlib.h>    // Untuk fungsi exit(), malloc(), dll

// Variabel global untuk menyimpan dua angka input
double num1, num2;

// Variabel global untuk menyimpan hasil tiap operasi
double hasil_jumlah, hasil_kurang, hasil_kali, hasil_bagi;

// Fungsi yang akan dijalankan oleh thread untuk penjumlahan
void* tambah(void* arg) {
    hasil_jumlah = num1 + num2;
    printf("Thread Penjumlahan Selesai (%.2f + %.2f = %.2f)\n", num1, num2, hasil_jumlah);
    pthread_exit(NULL); // Mengakhiri thread ini
}

// Fungsi untuk pengurangan
void* kurang(void* arg) {
    hasil_kurang = num1 - num2;
    printf("Thread Pengurangan Selesai (%.2f - %.2f = %.2f)\n", num1, num2, hasil_kurang);
    pthread_exit(NULL);
}

// Fungsi untuk perkalian
void* kali(void* arg) {
    hasil_kali = num1 * num2;
    printf("Thread Perkalian Selesai (%.2f * %.2f = %.2f)\n", num1, num2, hasil_kali);
    pthread_exit(NULL);
}

// Fungsi untuk pembagian
void* bagi(void* arg) {
    if (num2 != 0)
        hasil_bagi = num1 / num2;
    else {
        printf("Peringatan: Pembagian dengan nol!\n");
        hasil_bagi = 0;
    }
    printf("Thread Pembagian Selesai (%.2f / %.2f = %.2f)\n", num1, num2, hasil_bagi);
    pthread_exit(NULL);
}
```

```

int main() {
    pthread_t t1, t2, t3, t4; // Deklarasi 4 thread untuk 4 operasi

    // Input dari user
    printf("=== Kalkulator Paralel ===\n");
    printf("Masukkan dua angka: ");
    scanf("%lf %lf", &num1, &num2);

    // Membuat thread untuk masing-masing operasi
    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    // Menunggu semua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    // Menampilkan hasil akhir
    printf("\n=== Hasil Akhir Setelah Semua Thread Selesai ===\n");
    printf("Penjumlahan : %.2f\n", hasil_jumlah);
    printf("Pengurangan : %.2f\n", hasil_kurang);
    printf("Perkalian   : %.2f\n", hasil_kali);
    printf("Pembagian    : %.2f\n", hasil_bagi);

    return 0;
}

```

Output Program:

```
pradana@Bagus:~$ nano kalkulator_paralel.c
pradana@Bagus:~$ gcc kalkulator_paralel.c -o kalkulator_paralel
-lpthread
pradana@Bagus:~$ ./kalkulator_paralel
=== Kalkulator Paralel ===
Masukkan dua angka: 7 9
Thread Penjumlahan Selesai (7.00 + 9.00 = 16.00)
Thread Pengurangan Selesai (7.00 - 9.00 = -2.00)
Thread Perkalian Selesai (7.00 * 9.00 = 63.00)
Thread Pembagian Selesai (7.00 / 9.00 = 0.78)

=== Hasil Akhir Setelah Semua Thread Selesai ===
Penjumlahan : 16.00
Pengurangan : -2.00
Perkalian : 63.00
Pembagian : 0.78
pradana@Bagus:~$ |
```

Program Kalkulator Paralel berhasil menjalankan empat operasi aritmetika dasar, yaitu penjumlahan, pengurangan, perkalian, dan pembagian dengan menggunakan empat thread yang bekerja secara paralel. Setiap thread bertanggung jawab pada satu operasi sehingga proses perhitungan dapat berjalan secara bersamaan (concurrent), bukan secara berurutan. Hasil perhitungan ditampilkan setelah semua thread selesai dieksekusi menggunakan fungsi `pthread_join()`. Urutan kemunculan hasil setiap thread dapat berbeda-beda setiap kali program dijalankan, karena masing-masing thread berjalan secara independen dan dijadwalkan oleh sistem operasi. Secara keseluruhan, program ini menunjukkan bahwa penggunaan multithreading dapat meningkatkan efisiensi eksekusi dan memperlihatkan konsep dasar komputasi paralel dengan cara sederhana.

Tugas 2

Source Kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

// Struktur untuk menyimpan data yang akan diproses tiap thread
struct ThreadData {
    char *text;
    int start;
    int end;
    int word_count;
};

// Fungsi untuk menghitung jumlah kata pada bagian tertentu dari teks
void *count_words(void *arg) {
    struct ThreadData *data = (struct ThreadData *)arg;
    int count = 0;
    int in_word = 0;

    for (int i = data->start; i < data->end; i++) {
        if (data->text[i] == ' ' || data->text[i] == '\n' || data->text[i] == '\t')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            count++;
        }
    }

    data->word_count = count;
    pthread_exit(NULL);
}

// Fungsi untuk menghitung kata secara single-thread
int count_words_single(char *text, int length) {
    int count = 0;
    int in_word = 0;

    for (int i = 0; i < length; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
        }
    }

    return count;
}
```

```

        count++;
    }
}
return count;
}

```

```

int main() {
    FILE *file;
    char *buffer;
    long file_size;

    // Buka file teks yang akan dibaca
    file = fopen("input.txt", "r");
    if (!file) {
        perror("Gagal membuka file");
        return 1;
    }

    // Hitung ukuran file
    fseek(file, 0, SEEK_END);
    file_size = ftell(file);
    rewind(file);

    // Alokasikan memori dan baca isi file
    buffer = (char *)malloc(file_size + 1);
    fread(buffer, 1, file_size, file);
    buffer[file_size] = '\0';
    fclose(file);

    // ----- SINGLE THREAD -----
    clock_t start_single = clock();
    int total_words_single = count_words_single(buffer, file_size);
    clock_t end_single = clock();
    double time_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;

    // ----- MULTI THREAD (2 thread) -----
    pthread_t threads[2];
    struct ThreadData thread_data[2];

    int mid = file_size / 2;

    // Bagian 1
    thread_data[0].text = buffer;
    thread_data[0].start = 0;
    thread_data[0].end = mid;

```

```

// Bagian 2
thread_data[1].text = buffer;
thread_data[1].start = mid;
thread_data[1].end = file_size;

clock_t start_multi = clock();

// Jalankan dua thread
pthread_create(&threads[0], NULL, count_words, &thread_data[0]);
pthread_create(&threads[1], NULL, count_words, &thread_data[1]);

// Tunggu kedua thread selesai
pthread_join(threads[0], NULL);
pthread_join(threads[1], NULL);

int total_words_multi = thread_data[0].word_count + thread_data[1].word_count;
clock_t end_multi = clock();
double time_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

// ----- HASIL -----
printf("=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (Single Thread): %d\n", total_words_single);
printf("Jumlah kata (Multi Thread): %d\n", total_words_multi);
printf("Waktu eksekusi Single Thread: %.6f detik\n", time_single);
printf("Waktu eksekusi Multi Thread: %.6f detik\n", time_multi);

free(buffer);
return 0;
}

```

Output Program:

```

pradana@Bagus:~$ nano file_processing.c
pradana@Bagus:~$ gcc file_processing.c -o file_processing -lpthread
pradana@Bagus:~$ ./file_processing
=== HASIL PERHITUNGAN ===
Jumlah kata (Single Thread): 13
Jumlah kata (Multi Thread): 14
Waktu eksekusi Single Thread: 0.000004 detik
Waktu eksekusi Multi Thread: 0.005427 detik
pradana@Bagus:~$ |

```

Output program ini menunjukkan jumlah kata dari file teks pakai dua cara: single-thread dan multi-thread. Jumlah katanya sama, cuma waktu kerjanya beda. Biasanya, multi-thread lebih cepat karena dua thread kerja barengan hitung bagian file masing-masing, lalu hasilnya digabung. Jadi, output ini nunjukin seberapa efisien thread saat proses baca file.

Tugas 3

Source Kode:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10 // Maksimal 10 item di buffer

int buffer[BUFFER_SIZE];
int count = 0; // Jumlah item dalam buffer
int in = 0, out = 0; // Indeks untuk producer dan consumer

pthread_mutex_t mutex; // Mutex untuk sinkronisasi
pthread_cond_t full; // Kondisi jika buffer penuh
pthread_cond_t empty; // Kondisi jika buffer kosong

// Fungsi producer: menghasilkan angka random dan memasukkannya ke buffer
void *producer(void *arg) {
    while (1) {
        int item = rand() % 100; // Angka random 0-99

        pthread_mutex_lock(&mutex);

        // Tunggu jika buffer penuh
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&empty, &mutex);
        }

        // Masukkan item ke buffer
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;

        printf("Producer menghasilkan: %d (buffer: %d item)\n", item, count);

        pthread_cond_signal(&full); // Beri sinyal ke consumer
        pthread_mutex_unlock(&mutex);

        sleep(1); // Simulasi waktu produksi
    }
}

// Fungsi consumer: mengambil angka dari buffer dan memprosesnya
void *consumer(void *arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Tunggu jika buffer kosong
```

```

while (count == 0) {
    pthread_cond_wait(&full, &mutex);
}

// Ambil item dari buffer
int item = buffer[out];
out = (out + 1) % BUFFER_SIZE;
count--;

printf("Consumer memproses: %d (buffer: %d item)\n", item, count);

pthread_cond_signal(&empty); // Beri sinyal ke producer
pthread_mutex_unlock(&mutex);

sleep(2); // Simulasi waktu konsumsi
}
}

int main() {
    pthread_t prod_thread, cons_thread;

    srand(time(NULL)); // Inisialisasi random

    // Inisialisasi mutex dan kondisi
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&full, NULL);
    pthread_cond_init(&empty, NULL);

    // Buat thread producer dan consumer
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // Tunggu kedua thread (program jalan terus)
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    // Bersihkan resource
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&full);
    pthread_cond_destroy(&empty);

    return 0;
}

```

Output:

```
pradana@Bagus:~$ nano producer_consumer.c
pradana@Bagus:~$ gcc producer_consumer.c -o producer_consumer -lpthread
pradana@Bagus:~$ ./producer_consumer
Producer menghasilkan: 47 (buffer: 1 item)
Consumer memproses: 47 (buffer: 0 item)
Producer menghasilkan: 97 (buffer: 1 item)
Consumer memproses: 97 (buffer: 0 item)
Producer menghasilkan: 23 (buffer: 1 item)
Producer menghasilkan: 14 (buffer: 2 item)
Consumer memproses: 23 (buffer: 1 item)
Producer menghasilkan: 35 (buffer: 2 item)
^C
pradana@Bagus:~$ |
```

Proses producer dan consumer berjalan bergantian. Producer membuat angka acak dan memasukkannya ke buffer. Consumer mengambil angka dari buffer dan memprosesnya. Jumlah item di buffer naik saat producer menambah data dan turun saat consumer mengambilnya. Proses ini berlangsung sampai 20 angka selesai diproses. Setelah itu, program menampilkan pesan selesai ha ha.